

Project 1, Session 1: Multiplex assays of variant effect

- Project logistics
- DMS/MAVE experiments
- Interpreting experimental data
- Tabular data analysis with pandas

Logistics

Projects will provide a guided exploration of more long-term, real-world computational analyses. Along the way, we'll also supplement the Python foundation that we've been developing.

Each class period will end with an assignment to be completed outside of class, which we will discuss in the following class. We'll work in teams of four (except for one group of five). Each team member will have one of four roles describing their main (but **not only**) responsibility.

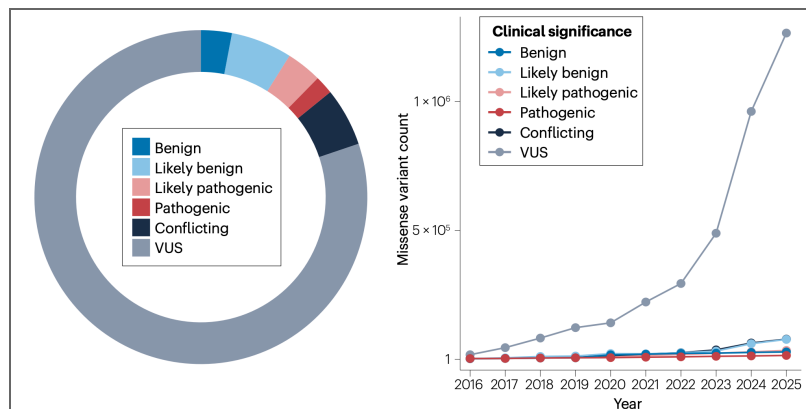
- **Biological lead:** understanding the biological question, assumptions, and interpretation
- **Computational lead:** overall computational design and approach
- **Verifier:** develops tests and controls, investigates failures
- **Integrator:** connects the pieces, organizes explanations

Teams work together to produce **one set of code** -
- all can contribute but there should be one final
product.

Genetic variation is ubiquitous but challenging to interpret

Germline variation in living humans likely covers all possible single nucleotide mutations that are compatible with life.

Mutations can have many effects, affecting the expression, activity, or regulation of proteins critical for life and health.

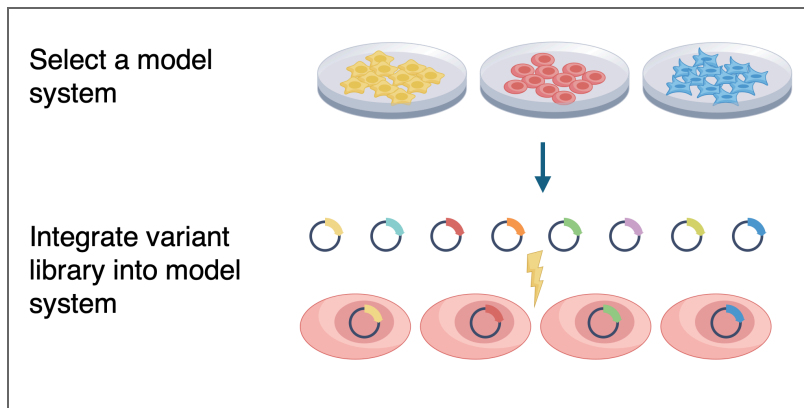


Why is this important? What could we learn from the effects of mutations? Is this limited to human proteins?

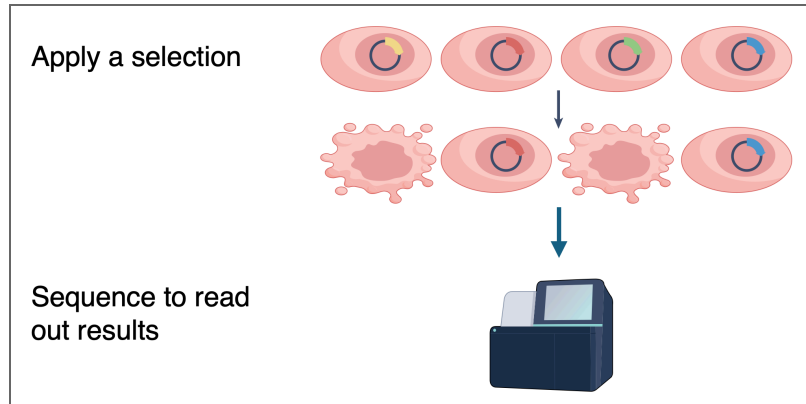
Reference: McEwen et al., Nat. Rev. Genet. 2026

Multiplex assays of variant effects (MAVEs) / deep mutational scanning (DMS)

In the last 10-15 years, a new family of MAVE/DMS methods have been developed to experimentally measure the effects of mutations in high throughput. Broadly, most take a similar approach.



Multiplex assays of variant effects (MAVEs) / deep mutational scanning (DMS)



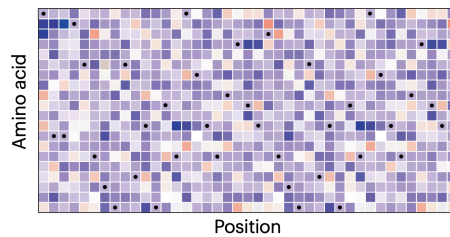
Multiplex assays of variant effects (MAVEs) / deep mutational scanning (DMS)

Convert
sequence data to
functional scores

...ACTGCGA... ...ACTTCGA...
...ACTGCGA... ...ACTTCGA...
...ACTGCGA... ...ACTTCGA...

Wild-type count = 3

Variant count = 2



Our first project

We will study a paper that developed and applied a new MAVE approach to study PTEN, a tumor suppressor protein.

PTEN mutations are commonly observed in many cancers. We'd like to understand how different mutations affect PTEN function.

For our purposes, this project will also provide an opportunity to design, code, and test analyses to answer an important biological question.

VAMP-seq

ARTICLES

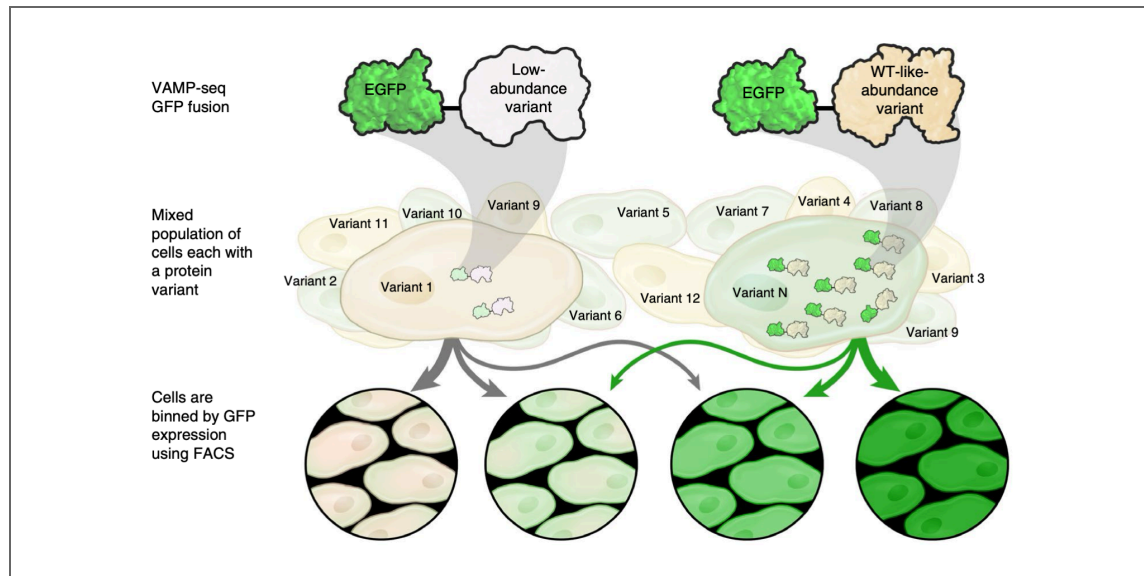
<https://doi.org/10.1038/s41588-018-0122-z>

nature
genetics

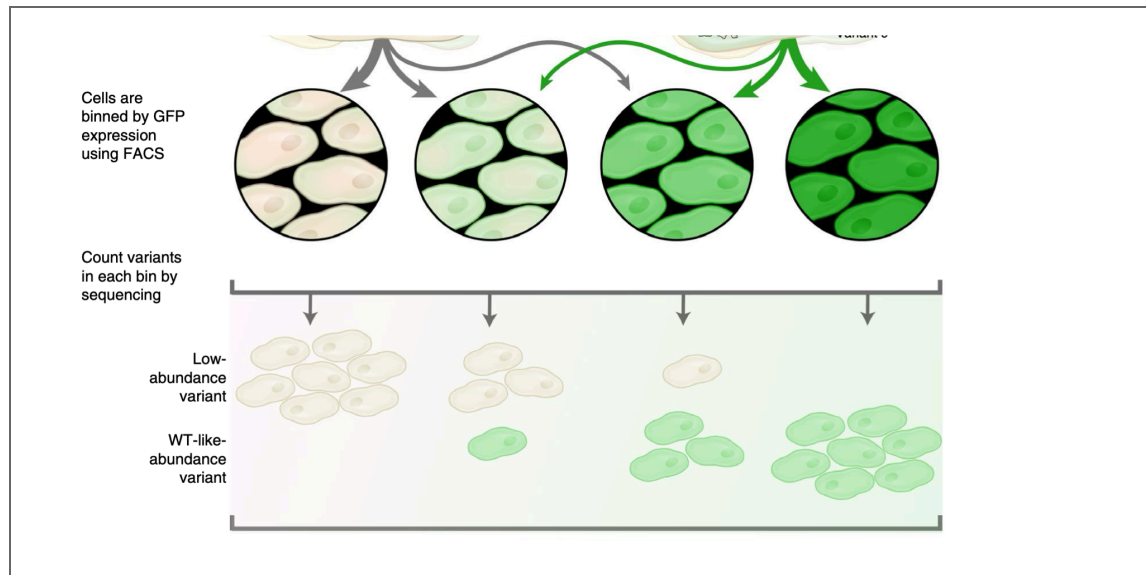
Multiplex assessment of protein variant abundance by massively parallel sequencing

Kenneth A. Matreyek ^{1,8}, Lea M. Starita^{1,8}, Jason J. Stephany¹, Beth Martin ¹, Melissa A. Chiasson¹,
Vanessa E. Gray¹, Martin Kircher ¹, Arineh Khechaduri¹, Jennifer N. Dines², Ronald J. Hause¹,
Smita Bhatia³, William E. Evans ⁴, Mary V. Relling⁴, Wenjian Yang⁴, Jay Shendure^{1,5*} and
Douglas M. Fowler ^{1,6,7*}

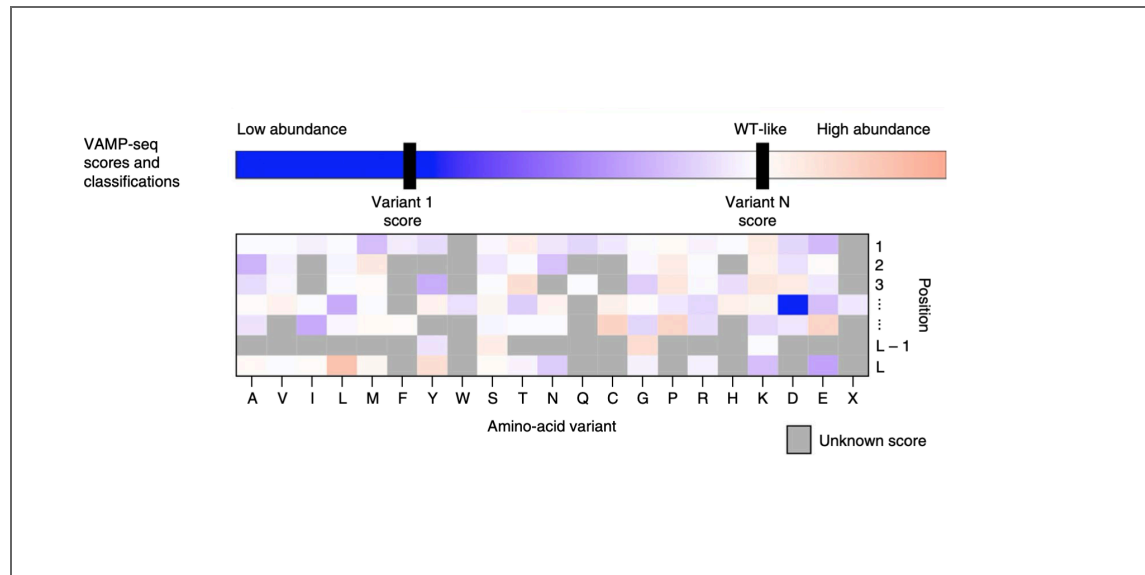
VAMP-seq overview: generate variants



VAMP-seq overview: bin variants by fluorescence



VAMP-seq overview: score abundance for each variant



A substitution can affect several properties

PTEN helps restrain cell growth signaling.
What could change when one amino acid changes?

Abundance • catalytic activity • localization • interactions

Which of these would an abundance measurement address directly?

What does the assay observe?

Reporter fluorescence → ordered bins → variant counts → score

What information would you need to describe one variant's distribution?

Four bin counts, the bin order, and the replicate number.

The reporter provides evidence about **abundance in this assay**.

Read an artificial library

One row = one variant in one replicate.

Format: p.His75Asp means that a histidine (H) at site 75 in the protein is replaced by aspartate (D).

WT represents the wild-type or reference sequence. Ter denotes a stop codon/nonsense mutation.

Variant, replicate 1	Bin 1	Bin 2	Bin 3	Bin 4
WT	9	42	267	376
p.Trp18Ter	221	110	41	6
p.Lys50Arg	0	4	62	320

Equal counts can represent unequal frequencies

	Bin 1	Bin 2
A variant's count	10	10
All variants' counts	100	1,000

Is this variant equally represented in the two bins?

$$F(v, b) = \frac{\text{count}(v, b)}{\sum_u \text{count}(u, b)}$$

Use one bin and one replicate at a time.

Compress frequencies across bins into one score

Bin weights: **0.25, 0.50, 0.75, 1.00**

For frequencies $[0, 0.1, 0.2, 0.1]$, we can compute a weighted score

$$W = \frac{0.25F_1 + 0.50F_2 + 0.75F_3 + 1.00F_4}{F_1 + F_2 + F_3 + F_4}$$

Do these four frequencies have to sum to one?

The same score can hide different distributions

Imagine one variant with most of its weight in the middle bins and another split between the extremes.

Could they have the same weighted mean? If so, would you interpret them in the same way?

An example function contract for computing the weighted score

`weighted_bin_score(frequencies)`

- Input: four finite, nonnegative frequencies in bin order.
- Output: one weighted score; input unchanged.
- Reject a zero total.

pandas

pandas provides a fast, flexible interface for working with “relational” or “labeled” data that is easy and intuitive

pandas can read and store compressed data automatically, which is a **big deal**

Especially useful for large data sets with **mixed types**

Example: the `iris` dataset in `seaborn`

Data in `pandas` is stored in `DataFrame` objects that can be manipulated in many ways

`iris` stores measurements of different flower species

`matplotlib` and `seaborn` are popular libraries for plotting

```
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plot
import seaborn as sns

df = sns.load_dataset('iris')
df.head(10)
```

Basic data frame checks

DataFrame: a table with named columns. **Series:** one named column of values.

```
df.columns
```

```
df['petal_length']
```

```
len(df)
```

No (default) access by number

```
df[0]
```

.iloc : Position indexing

If you want to reference a pandas data frame with position based indexing, use `.iloc` - works just like `numpy`

```
df.iloc[3, 1]
```

```
df.iloc[0,:]
```

Pandas uses NaN to indicate missing data

.loc : Label indexing

You can also index by the label names

Note that rows are indexed by their named index

```
df.loc[:3, 'petal_width']
```

```
df.loc[[0,3,5],['petal_length','sepal_length']]
```

Row slices

```
df[10:20]
```

Note: `.loc` indices are *absolute*, not *relative*

```
df[10:20].iloc[[0,3,5]]
```

```
df[10:20].loc[[0,3,5]]
```

Sorting

```
df.sort_values(by="petal_length").head(3)
```

Showing the relationships between variables

How are petal and sepal lengths related?

```
sns.jointplot(data=df, x='petal_length',  
y='sepal_length');
```

Selecting subsets

```
print(np.unique(df.species))
```

Is the relationship between petal and sepal length different for different species?

```
df_setosa      = df[df.species=='setosa']  
df_versicolor = df[df.species=='versicolor']  
df_virginica   = df[df.species=='virginica']  
  
sns.jointplot(data=df_setosa, x='petal_length',  
y='sepal_length');
```

Visualizing multiple variables at once

Because the data is stored in a simple format, it is easy to quickly relate many variables

```
sns.pairplot(data=df, hue='species');
```

Selecting observations that meet multiple conditions

Which flowers have a sepal length > 7 and a petal length > 5 ?

```
example = df.loc[(df['sepal_length'] > 7) &  
(df['petal_length'] > 5)]  
example
```

Calculate a new quantity for every row

```
df['sepal_length_to_width_ratio'] = df['sepal_length']  
/ df['sepal_width']  
df[['species', 'sepal_length', 'sepal_width',  
    'sepal_length_to_width_ratio']].head(10)
```

Will this assignment overwrite the source file?

To save the dataframe with the new column as a file, we can use methods like `to_csv`.

Summarize data by group

groupby allows us to group rows of the dataframe together based on their value in one column

agg lets us aggregate statistics across groups

```
summary = df.groupby('species')[['sepal_length',  
'petal_length']].agg(["count", "mean"])  
summary
```

Assignment for next time

Before working with the real data, we will start with a toy VAMP-seq dataset. With your team, download this dataset and perform the following analyses:

- Explain what each column means
- Compute weighted scores for each variant
- Compute normalized scores for each variant by 1) subtracting the median nonsense score from each variant, then 2) dividing by the wild-type score (with the median nonsense score subtracted)
- Think critically about the results

For each analysis, be prepared to explain:

- What you did, conceptually (in words/figures)
- How the code that you developed achieved this goal
- How you checked your assumptions and verified that the code was doing what you intended